

Homework 4: Regression & Multi-Class Classification

In this assignment, you will work with a synthetic Human Resources Information System (HRIS) dataset. The dataset simulates employee-level information and includes both behavioral and structural predictors of job outcomes. Your goal is to build predictive models for both a continuous outcome and a multi-class categorical outcome.

You must submit a Jupyter Notebook containing all code, visualizations, and written explanations.

Dataset Description

The dataset contains 97,500 employees and includes the following variables:

- `department_code`: Department identifier (1–12).
- `age_years`: Employee age (18–65).
- `tenure_months`: Tenure in months (derived from `tenure_years`).
- `tenure_years`: Years employed at the organization (continuous, 0–30).
- `overtime_hours_month`: Average overtime hours per month (0–60).
- `remote_work_ratio`: Proportion of time working remotely (0–1).
- `job_level`: Job level in the organization (integer, 1–8).
- `goal_completion_rate`: Proportion of annual goals completed (0–1).
- `engagement_score`: Employee engagement score (1–100).
- `job_satisfaction`: Job satisfaction score (1–100).
- `absences_days_12mo`: Number of absence days in the past 12 months (0–30).
- `engagement_percent`: Engagement score converted to proportion (0–1).
- `commute_minutes`: Average daily commute time in minutes (5–120).
- `training_hours_12mo`: Training hours completed in the past 12 months (0–120).
- `team_size`: Size of the employee's team (2–25).
- `last_promotion_years_ago`: Years since last promotion (0–15).
- `performance_rating`: Performance appraisal rating (1 = lowest, 5 = highest).

Part 1 – Predicting Job Satisfaction (Regression)

Your first objective is to predict the continuous outcome variable: `job_satisfaction`.

You may use any regression model (e.g., Linear Regression, Ridge, Lasso, Decision Trees, Random Forests etc.).

Steps:

- Split the data into training and test sets or use cross-validation.
- Train your regression model. Name this model “regressor”.

- Report training and test R^2 . (We haven't covered this in class but there are plenty of methods to calculate this metric including `r2_score` from `sklearn.metrics` or using `.score` method from your model).
- Identify which features are most predictive of the target.

Visualization and Interpretation:

Identify the top 2 most predictive features. For these features create two separate graphs each containing:

- A scatterplot of the feature against `job_satisfaction`
- Overlay the fitted regression line

Explain why these features are predictive and whether the relationships make conceptual sense.

Part 2 – Predicting Performance Rating (Multi-Class Classification)

Your second objective is to predict the 5-class categorical target: `performance_rating`.

You may choose any classification model, including:

- K-Nearest Neighbors (KNN)
- Logistic Regression (multinomial)
- Decision Trees
- Ensemble methods including Random Forests

Steps:

- Split the data into training and test sets or use cross-validation. It is recommended to stratify by target.
- Train your classification model. Name your model “classifier”.
- Report training and test weighted f1 scores.
- Identify which features are most predictive of the target. This looks different based on the model/algorithm you chose.

Decision Boundary Visualization and Interpretation:

Using your top 3 predictive features, generate decision boundary plots for all 3 possible feature pairs (Feature A vs B, A vs C, B vs C).

Explain the following:

- Why these features are predictive
- What the decision boundaries look like
- Whether the boundaries make conceptual sense in the HR context

Performance Requirement:

Your model must achieve the test $f1$ / R^2 threshold posted on Canvas.

Final grading will use a hidden evaluation dataset. **Models that overfit may perform poorly on the hidden evaluation set.**

Model Validation & Overfitting:

You are strongly encouraged to use validation techniques such as cross-validation, regularization, pruning, or hyperparameter tuning.

Be aware that some algorithms (e.g., deep trees, high KNN sensitivity, complex ensembles) can overfit more easily.

Grading (140 points total):

- Data Preparation and Pre-processing (5 points)
- Model Implementation – Regression Task (5 points)
- **Model Evaluation – Regression Task (20 points)***
- Visualization and Interpretation – Regression Task (10 points)
- Model Implementation – Classification Task (20 points)
- **Model Evaluation – Classification Task (60 points)***
- Model Visualization and Interpretation – Classification Task (20 points)

* For each evaluation score, 50% is based on achieving evaluation metric on training set and 50% is based on achieving evaluation metric on hidden test set.

Submission Instructions

Submit a Jupyter Notebook (.ipynb) and note:

- Ensure you use **random seed value of 2026** for all models and computations where appropriate.
- Clean and organize your code
- **The final model's name for part 1 should be "regressor" and for part 2 should be "classifier". This is used for grading.**
- All visualizations included and rendered
- Clearly label outputs
- Include written explanations for both parts

Your notebook should run from top to bottom without errors.

Additional Guidelines

- The first task (regression) is significantly easier than second (classification). Ensure you're budgeting your time accordingly.
- We use weighted f1 score because our classes are heavily imbalanced. There are many ways you can get this score, but one way is using `f1_score` function from `sklearn.metrics` and passing `average="weighted"` to it. You can find more by reading the documentation.
- To ensure your model is not overfitting you should employ some type of validation. Usually when n/p ratio is sufficiently large (where n is sample size and p is number of features), using a simple train/test split should suffice but cross-validation is also a solid strategy. [Read more here.](#)
- This homework is designed to get you comfortable with reading some of online documentations. You are encouraged to choose a model that makes the most sense given the nature of your data. Hyperparameter fine-tuning should always improve performance (especially to prevent overfitting) but is not necessarily the focus on this homework. But if you are confident in your choice of model you can invest in fine-tuning as well.
- If you find yourself spending time tweaking model parameters a lot, a good idea would be investing in a formal search strategy. A grid or randomized search is recommended although is strictly not required for successful performance on this homework. [Read more here.](#)
- With good use of the provided data and data preprocessing and good choice of model you should perform successfully on this homework. But you may **explore additional models** that might even significantly beat the set thresholds. You have a wide range of choice of models from SVM to other boosted models such as `HistGradientBoostingClassifier` or other ensemble trees such as `ExtraTreesClassifier`. You should provide some explanation for why this model makes sense, but you may get **extra credit** for beating our thresholds by a large margin.